

Linear regression and gradient descent

stochastic gradient descent

Linear regression is about finding the line of best fit for a dataset. This line can then be used to make predictions.

Gradient descent is a tool to arrive at the line of best fit

Statistics way of computing line of best fit:

A line can be represented by the formula:

$$y = mx + b.$$

The formula for slope m of the regression line is:

$$m = r * (SD \text{ of } y / SD \text{ of } x)$$

r = correlation coefficient between x and y values (r).

After calculation of m .

Put the value of x as average value of x . and y as average value of y .

Value of b can be calculated.

For example $m=1.2$ and $b=-12.87$

Regression line using statistics is $y = 1.2x - 12.87$

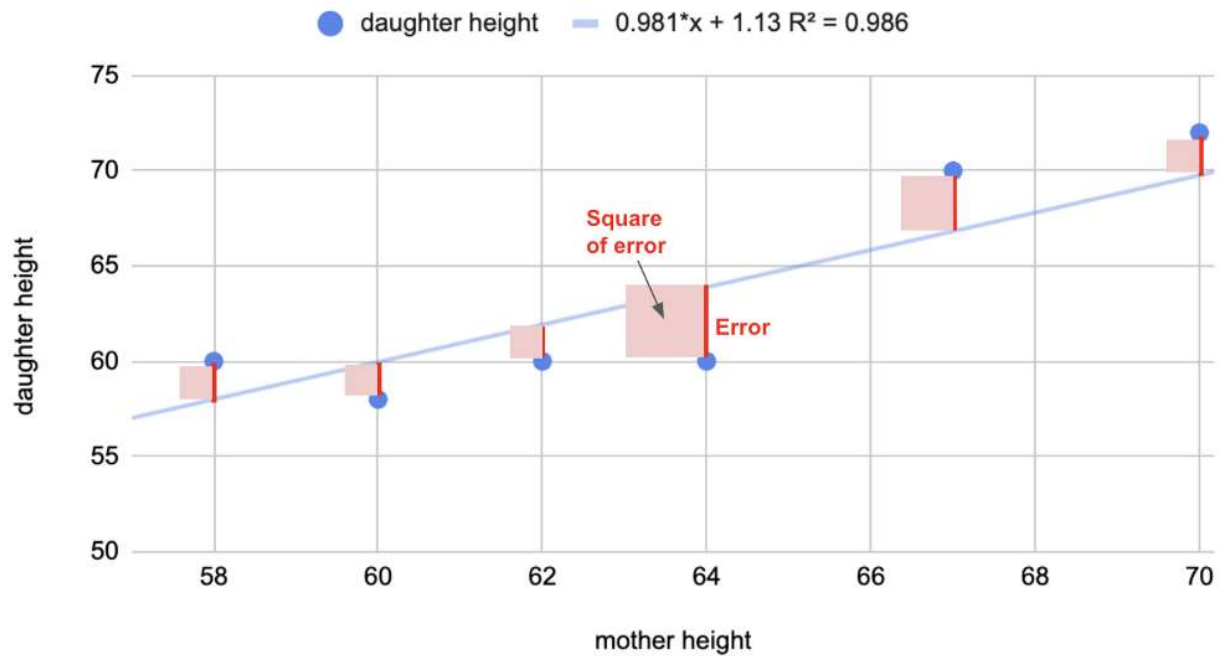
Gradient descent way of computing line of best fit:

In gradient descent, you start with a random line. Then you change the parameters of the line (i.e. slope and y-intercept) little by little to arrive at the line of best fit.

How do you know when you arrived at the line of best fit?

For every line you try — line A, line B, line C, etc — you calculate the sum of squares of the errors. If line B has a smaller value than line A, then line B is a better fit, etc.

daughter height vs. mother height

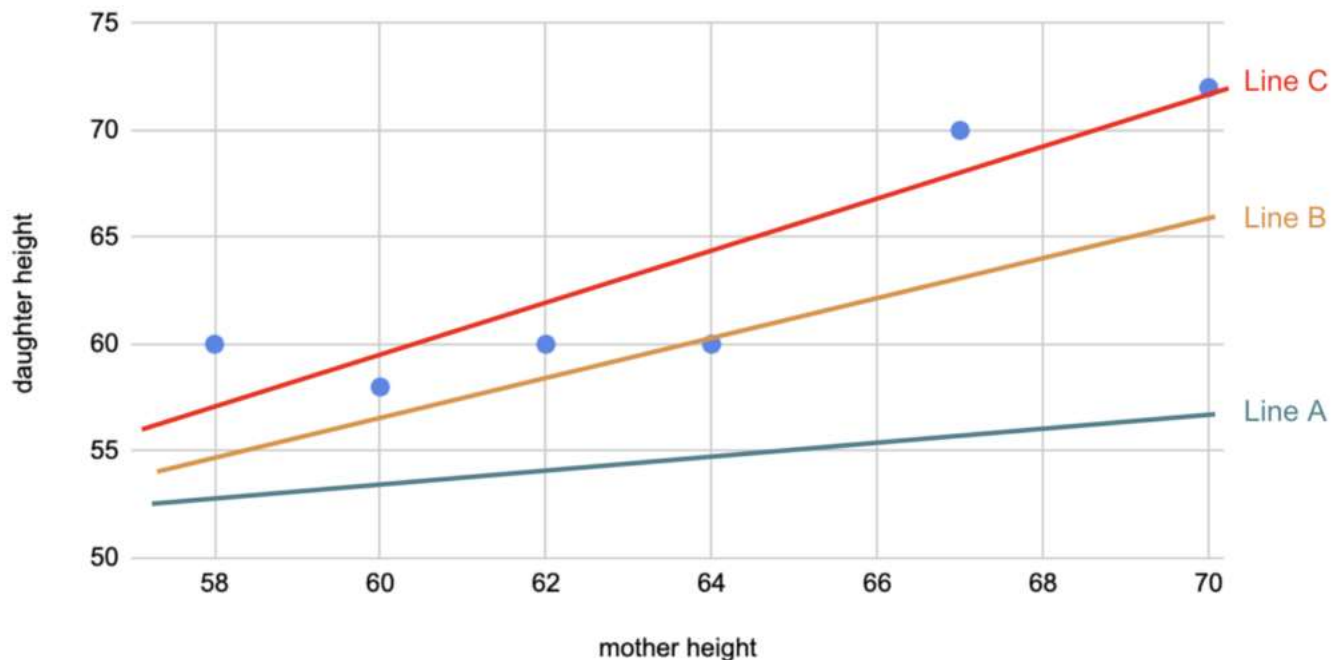


Error is your actual value minus your predicted value. The line of best fit minimizes the sum of the squares of all the errors. In linear regression, the line of best fit we computed above using correlation coefficient also happens to be the least squared error line. That's why the regression line is called the **LEAST SQUARE REGRESSION LINE**.

Line of best fit is the least square regression line

In the image below, line C is better fit than line B, which is better fit than line A.

daughter height vs. mother height



Line C is better fit than B, which is better fit than A

At a high level, this is how gradient descent works:

You start with a random line, let's say line A. You compute the sum of squared errors for that line. Then, you adjust your slope and y-intercept. You compute the sum of squared errors again for your new line. You continue adjusting until you reach a local minimum, where the sum of squared errors is the smallest and additional tweaks does not produce better result. The way you adjust your slope and intercept will be covered in more details momentarily.

Gradient descent is an algorithm that approaches the least squared regression line via minimizing sum of squared errors through multiple iterations.

Mathematics of Gradient Descent Algorithm

In machine learning terminology, the sum of squared error is called the “cost”. This cost equation is:

$$\text{cost} = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

cost equation

Where:

$h(x^{(i)})$	= the predicted value of regression line
$y^{(i)}$	= the actual value from dataset
$(h(x^{(i)}) - y^{(i)})^2$	= one error squared
m	= number of data points

This equation is therefore roughly “sum of squared errors” as it computes the sum of predicted value minus actual value squared.

The $1/2m$ is to “average” the squared error over the number of data points so that the number of data points doesn’t affect the function.

In gradient descent, the goal is to minimize the cost function.

We do this by trying different values of slope and intercept. But which values to try and how do you go about changing those values?

We change their values according to the gradient descent formula, which comes from taking the partial derivative of the cost function.

By taking the partial derivative, you arrive at the formula:

$$\begin{aligned}\text{new_intercept} &= \text{old_intercept} - \alpha \cdot \left(\frac{1}{m}\right) \sum (h(x^{(i)}) - y^{(i)}) \\ \text{new_slope} &= \text{old_slope} - \alpha \cdot \left(\frac{1}{m}\right) \sum (h(x^{(i)}) - y^{(i)}) \cdot x^{(i)}\end{aligned}$$

Gradient descent formula by taking partial derivative of the cost function

This formula computes by how much you change your theta with each iteration.

The alpha (α) is called the learning rate. The learning rate determines how big the step would be on each iteration. It's critical to have a good learning rate because if it's too large your algorithm will not arrive at the minimum, and if it's too small, your algorithm will take forever to get there. For my example, I picked the alpha to be 0.001

Gradient descent is an algorithm that approaches the least squared regression line via minimizing sum of squared errors through multiple iterations.

example:

House size & price Data:

House Size sq.ft (X)	House Price (Y)
2100	199,000
2400	219,000
2425	245,000
2550	255,000
2600	279,000
2700	308,000
2700	312,000
2875	319,000
3350	324,000
3450	405,000

1.Scale the data using standard scaler or Min max scaler.

Normalized data.

House Size sq.ft (X new)	House Price (Y new)
0	0
0.2222222222	0.097087379
0.240740741	0.223300971
0.3333333333	0.27184466
0.37037037	0.388349515
0.4444444444	0.529126214
0.4444444444	0.548543689
0.574074074	0.582524272
0.925925926	0.606796117

Predicted value:-

$$\hat{y} = a + b * X \text{ new}$$

2. Let's start by initializing the random weights. a= 0.45, b=0.75

Random weight	
a	b
0.45	0.75

3. Compute the $\hat{y}$

Consider the random observation from the new scaled data

X new= 0.222222222 ,

Y new= 0.097087379

$$\hat{y} = a + b * X \text{ new}$$

$$\hat{y} = 0.45 + 0.75 * 0.222222222$$

$$= 0.61666$$

4. Compute loss

$$\text{Residual or Error} = \hat{y} - y \text{ new}$$

$$= 0.6166666 - 0.0970873$$

$$= 0.519579$$

Objective is to Minimize the error, with the update the best effective value of **a** & **b**.

5. Compute partial differentials

Calculate the partial derivative of the total error with respect to weight

The partial differentials are as follows:

The partial derivative of total error w.r.t. a (weight)

$$= \partial E / \partial a$$

$$= \partial (\hat{y} - y \text{ new}) / \partial a$$

$$= \partial(\hat{y} - (a + b * X_{\text{new}})) / \partial a$$

$$= -(\hat{y} - y_{\text{new}})$$

$$= -0.519579$$

The partial derivative of total error w.r.t. b (weight)

$$= \partial E / \partial b$$

$$= \partial(\hat{y} - y_{\text{new}}) / \partial b$$

$$= \partial(\hat{y} - (a + b * X_{\text{new}})) / \partial b$$

$$= -(\hat{y} - y_{\text{new}}) * X_{\text{new}}$$

$$= -0.519579 * 0.222222222$$

$$= -0.115462064$$

6. Update the weights

Update 'a'

Use a small learning rate (alpha=0.01)

$$\text{New a} = \text{old a} - \text{alpha} * \partial E / \partial a$$

$$= 0.45 - 0.01 * -0.519579$$

$$= 0.455196$$

Update 'b'

$$\text{New b} = \text{old b} - \text{alpha} * \partial E / \partial b$$

$$= 0.75 - 0.01 * -0.115462064$$

$$= 0.75115$$

We have just finished our first iteration of stochastic gradient descent and we have updated our weights to be **a = 0.455196** and **b = 0.455196**.

Repeat this process for remaining instances from our dataset.

7. Iteration: 5

Let's jump ahead. Below is a list of all the updated values for the coefficients over the 5 iterations.

	House Size (X new)	House Price (Y new)	Random weight			Error
			a	b	$\hat{y} = a + b \cdot X \text{ new}$	$\hat{y} - y \text{ new}$
			Updated Weight			
Iteration-1	0.22222222	0.097087379	0.45	0.75	0.616666667	0.519579
Iteration-2	0.240740741	0.223300971	0.455196	0.751155	0.636029313	0.412728
Iteration-3	0.333333333	0.27184466	0.459323	0.752148	0.710039152	0.438194
Iteration-4	0.37037037	0.388349515	0.463705	0.753609	0.742819419	0.35447
Iteration-5	0.444444444	0.529126214	0.46725	0.754922	0.802770487	0.273644
	0.444444444	0.548543689				
	0.574074074	0.582524272				
	0.925925926	0.606796117				